Algorytmy ewolucyjne

Martyna Sadowska, nr indeksu 333157

Wstęp

W ramach projektu zaimplementowano algorytm ewolucyjny wraz z różnymi algorytmami selekcji i mutacji. Został on zastosowany przy maksymalizacji funkcji, rozwiązywaniu problemu cutting stock oraz optymalizacji wag w sieci MLP. Przetestowano jego skuteczność w rozwiązywaniu zadanych problemów oraz zbadano wpływ różnych parametrów na jego działanie.

Implementacja podstawowego algorytmu ewolucyjnego

Na tym etapie został zaimplementowany podstawowy algorytm ewolucyjny wraz z funkcjami selekcji, mutacji i krzyżowania. Populacja jest inicjowana jako losowi osobnicy z rozkładu jednostajnego z zakresu wybranego przez użytkownika. Zasadnicza część algorytmu ma charakter iteracyjny - trening składa się z ustalonej z góry liczby generacji (pętli), podczas których nowa populacja jest poddawana czterem stałym krokom.

Pierwszym krokiem każdej generacji jest krzyżowanie osobników. Cała dotychczasowa populacja jest losowo parowana, a następnie zgodnie z określonym prawdopodobieństwem krzyżowania każda z par może zostać poddana krzyżowaniu. Jedyna funkcja krzyżowania wprowadzona na każdym z etapów projektu to krzyżowanie jednopunktowe - losowo wybierany jest punkt k, a dziecko powstaje jako pierwsze k elementów genu pierwszego rodzica oraz elementy od $(k+1)$-szego genu drugiego rodzica.

Drugim krokiem jest mutacja - każde z powstałych dzieci może zostać poddane wybranej mutacji zgodnie z określonym prawdopodobieństwem mutacji. Na tym etapie używana jest mutacja gaussowska - 5% wag osobnika zostaje przesuniętych o losową wartość z rozkładu normalnego o odchyleniu standardowym równym 0.1.

Trzecim krokiem jest ewaluacja, czyli wyliczenie wyniku funkcji celu dla każdego z osobników populacji.

Ostatnim krokiem jest selekcja, czyli wybranie najlepszych n osobników, które przejdą do następnej generacji, gdzie n jest wybieranym przez użytkownika rozmiarem populacji. Zaimplementowane zostały trzy algorytmy selekcji - rankingowa, turniejowa i ruletkowa.

- Selekcja rankingowa wybiera z całej populacji n osobników o najlepszych wartościach funkcji celu.
- W selekcji turniejowej przeprowadzanych jest n niezależnych turniejów. Turniej polega na wylosowaniu z pełnej populacji k osobników (zwykle 3), a następnie wybraniu z nich najlepszego osobnika. Dzięki temu jeden osobnik może zostać wybrany kilka razy do nowej populacji. Zapewnia to również (podobnie jak selekcja rankingowa), że osobnik z absolutnie najwyższym wynikiem nie zostanie odrzucony.
- Selekcja ruletkowa oznacza losowy wybór $n-1$ osobników, gdzie każdy z nich ma prawdopodobieństwo wprost proporcjonalne do jego przystosowania. Na tym etapie minimalizujemy funkcję celu, dlatego prawdopodobieństwo wyboru i-tego osobnika określa poniższy wzór:

  $$p_i = \frac{S_{\max} - S_i + \epsilon}{\sum_{j=1}^{n}(S_{\max} - S_j + \epsilon)},$$

  gdzie $S_{\max}, S_i$ oznaczają wartości funkcji celu - maksymalną oraz i-tego osobnika, a ϵ to bardzo mała stała (10^{-6}). Zabezpiecza ona przed dzieleniem przez zero oraz daje nawet najgorszemu osobnikowi mikroskopijną szansę na przeżycie, co sprzyja zachowaniu różnorodności genetycznej. Aby upewnić się, że w procesie losowania nie utracimy absolutnego lidera, najlepszy osobnik jest automatycznie kopiowany do nowej populacji (tzw. elitaryzm), dlatego kołem ruletki losujemy jedynie $n-1$ miejsc.

Po zakończeniu selekcji otrzymujemy nową populację. W tym momencie pojedyncza generacja dobiega końca, a cały czteroetapowy cykl (krzyżowanie, mutacja, ewaluacja, selekcja) rozpoczyna się od nowa dla nowej populacji. Proces ten powtarza się w pętli aż do spełnienia warunku stopu (osiągnięcia maksymalnej liczby generacji).

Na tym etapie testowany był wpływ parametrów algorytmu (rozmiar populacji, prawdopodobieństwa krzyżowania i mutacji, algorytm selekcji) na dwóch funkcjach: $x^2 + y^2 + 2z^2$ oraz pięciowymiarowej funkcji Rastrigina określonej wzorem:

$$f(\mathbf{x}) = An + \sum_{i=1}^{n}[x_i^2 - A\cos(2\pi x_i)],$$

gdzie A zostało ustalone jako 10 a n to liczba wymiarów równa 5. Dla obu funkcji szukane było globalne minimum, które jest w punkcie $\mathbf{x} = 0$ i wynosi $f(\mathbf{x}) = 0$. Dla pierwszej funkcji (nazywanej dalej funkcją trójwymiarową) populacja była inicjowana z zakresu $(-10, 10)$, natomiast dla Rastrigina, zgodnie z literaturą, z zakresu $(-5.12, 5.12)$.

Przeprowadzono testy dla siatki parametrów przedstawionej w tabeli 1. Każdy wariant uruchomiono 5-krotnie dla zapewnienia powtarzalności wyników. Każde wywołanie było wykonywane przez 100 generacji. Była to wartość, która podczas testów okazała się wystarczająca do uzyskania dobrych wyników. Pełne wyniki umieszczone są w tabeli 2 i na wykresach 1 dla funkcji trójwymiarowej oraz w tabeli 3 i na wykresach 2 dla funkcji Rastrigina. Przebieg generacji dla trzech najlepszych modeli dla każdego z algorytmów selekcji jest przedstawiony na wykresach 3 dla funkcji trójwymiarowej i na wykresach 4 dla funkcji Rastrigina.

Tabela 1: Siatka testowanych hiperparametrów algorytmu ewolucyjnego.

Hiperparametr	Testowane wartości
Metoda selekcji	Turniejowa, Rankingowa, Ruletkowa
Rozmiar populacji	50, 100, 200
Prawdopodobieństwo krzyżowania	0.5, 0.7, 0.9
Prawdopodobieństwo mutacji	0.1, 0.3, 0.5

Tabela 2: Średnie wyniki funkcji celu wraz z odchyleniem standardowym dla 5 wywołań algorytmu ewolucyjnego dla funkcji trójwymiarowej, gdzie n oznacza rozmiar populacji, a najlepsze rozwiązanie to wartość $\mathbf{x}$ dla wywołania o najlepszej wartości funkcji celu. Wyniki są uporządkowane rosnąco względem średniego wyniki, co oznacza, że im wyżej znajduje się dany zestaw parametrów, tym lepsze wyniki uzyskiwał.

	Selekcja	n	P-stwo krzyżowania	P-stwo mutacji	Średni wynik	Odch. std.	Najlepsze rozwiązanie
0	rankingowa	200	0.9	0.3	7.43e-07	3.95e-07	[0.000126, 0.000463, -6.9e-06]
1	rankingowa	200	0.5	0.5	1.01e-06	6.54e-07	[4.74e-06, 0.000253, 0.000453]
2	rankingowa	200	0.9	0.5	1.12e-06	5.98e-07	[9.94e-05, 0.000203, 0.000397]
3	rankingowa	200	0.7	0.5	1.24e-06	7.74e-07	[4.43e-05, -8.18e-05, 0.000261]
4	rankingowa	100	0.9	0.5	2e-06	1.29e-06	[0.000145, -6.81e-06, -1.53e-05]
5	turniejowa	200	0.5	0.3	2.09e-06	1.86e-06	[0.00054, 0.000111, 0.000177]
6	turniejowa	200	0.9	0.5	2.55e-06	2.03e-06	[-0.00032, 0.00062, 3.23e-05]
7	turniejowa	200	0.7	0.5	2.6e-06	2.7e-06	[0.000471, -0.000217, -0.000463]
8	turniejowa	200	0.7	0.3	3.22e-06	1.84e-06	[0.000274, 0.000171, 0.000576]
9	turniejowa	200	0.9	0.3	3.77e-06	2.87e-06	[0.00103, 0.000827, -0.000143]
10	turniejowa	200	0.5	0.5	4.43e-06	2.46e-06	[0.00106, 9.84e-05, -0.00039]
11	rankingowa	100	0.5	0.5	5.81e-06	1.19e-06	[-0.000346, 0.00173, -0.000546]
12	rankingowa	100	0.7	0.5	6.07e-06	5.15e-06	[-0.000283, -8.43e-05, -0.000564]
13	rankingowa	200	0.7	0.3	6.15e-06	5.91e-06	[-0.00011, -5.51e-05, 0.000579]
14	rankingowa	100	0.7	0.3	7.25e-06	6.41e-06	[0.000563, 0.00079, 0.000797]
15	turniejowa	100	0.9	0.5	7.85e-06	1.09e-05	[-1.15e-05, -0.000581, -0.000262]
16	rankingowa	200	0.5	0.3	9.21e-06	7.36e-06	[-6.88e-05, 0.000229, -0.000964]
17	rankingowa	200	0.5	0.1	1.05e-05	5.83e-06	[-0.00138, 0.000254, 0.00135]
18	rankingowa	100	0.9	0.3	1.16e-05	9.45e-06	[0.000885, 0.000193, -0.0006]
19	turniejowa	100	0.7	0.5	1.37e-05	1.23e-05	[0.000361, 6.86e-05, -0.000762]
20	turniejowa	200	0.7	0.1	1.58e-05	7.84e-06	[0.0011, -0.000945, 0.000159]
21	turniejowa	100	0.5	0.5	1.64e-05	1.62e-05	[0.00137, 0.000257, -0.000177]
22	rankingowa	100	0.5	0.3	1.71e-05	9.86e-06	[-0.00215, 0.000184, -0.000364]
23	ruletkowa	200	0.7	0.5	1.84e-05	1.41e-05	[-0.000914, -0.000723, 0.00102]
24	turniejowa	50	0.9	0.5	2.28e-05	1.82e-05	[0.00155, 0.000112, -0.000927]
25	rankingowa	200	0.7	0.1	2.5e-05	3.03e-05	[-0.00173, -0.000644, 0.00071]
26	turniejowa	100	0.9	0.3	2.52e-05	1.47e-05	[0.000671, -0.00102, -0.000615]
27	rankingowa	200	0.9	0.1	2.82e-05	4.14e-05	[-0.000862, 0.000677, 0.000101]
28	rankingowa	50	0.5	0.5	3.1e-05	2.33e-05	[2.33e-05, 0.00151, 0.00193]

Ciąg dalszy na następnej stronie...

Tabela 2 - ciąg dalszy z poprzedniej strony

	Selekcja	n	P-stwo krzyżowania	P-stwo mutacji	Średni wynik	Odch. std.	Najlepsze rozwiązanie
29	turniejowa	100	0.7	0.3	3.93e-05	6.76e-05	[2e-05, -0.000742, 0.000637]
30	ruletkowa	100	0.9	0.5	4.12e-05	4.79e-05	[-0.000594, 0.00119, 0.000213]
31	turniejowa	100	0.5	0.3	5.09e-05	3.44e-05	[-0.00196, -0.000498, -0.000244]
32	turniejowa	200	0.9	0.1	5.54e-05	5.71e-05	[0.000614, -0.000442, 0.00211]
33	ruletkowa	100	0.5	0.5	5.57e-05	5.14e-05	[0.00139, 3.27e-05, -0.000189]
34	ruletkowa	200	0.5	0.5	6e-05	5.31e-05	[1.71e-05, 0.00229, -0.00167]
35	ruletkowa	200	0.9	0.3	6.4e-05	3.22e-05	[-0.00205, -0.00192, 0.00147]
36	rankingowa	50	0.9	0.5	6.57e-05	5.38e-05	[-0.000628, 0.00124, 0.000925]
37	ruletkowa	200	0.7	0.3	8.31e-05	5.95e-05	[0.00011, -0.000258, 0.00135]
38	ruletkowa	200	0.9	0.5	8.91e-05	8.98e-05	[0.000142, -0.00324, 0.000247]
39	rankingowa	100	0.7	0.1	9.34e-05	6.3e-05	[0.000922, -0.000105, 0.00165]
40	turniejowa	50	0.9	0.3	9.86e-05	0.000158	[0.00111, -0.00241, -0.0004]
41	rankingowa	100	0.9	0.1	0.000102	9.11e-05	[0.00241, -0.00101, 0.000414]
42	turniejowa	50	0.7	0.5	0.000107	0.00013	[-0.00408, -0.00202, 0.000929]
43	turniejowa	200	0.5	0.1	0.000126	7.71e-05	[-0.00185, 0.00057, -0.00169]
44	ruletkowa	100	0.7	0.5	0.000127	9.11e-05	[0.00026, -0.000472, 0.00208]
45	rankingowa	50	0.9	0.3	0.000134	0.000159	[-0.00248, -0.000235, 0.00147]
46	ruletkowa	200	0.5	0.1	0.000148	0.000113	[-0.00133, 0.00189, -0.00124]
47	ruletkowa	50	0.7	0.5	0.00016	0.000159	[0.000293, 0.0046, -0.000575]
48	ruletkowa	100	0.7	0.3	0.000165	0.000109	[0.000297, 0.00277, -0.00118]
49	ruletkowa	50	0.5	0.5	0.000235	0.000247	[0.000849, 0.00219, 0.00174]
50	ruletkowa	100	0.9	0.1	0.000249	0.000255	[0.00524, -0.00786, -9.02e-05]
51	ruletkowa	50	0.7	0.3	0.000254	0.000191	[-0.00388, 0.00176, 0.000661]
52	ruletkowa	50	0.9	0.3	0.000269	0.000281	[-0.00214, -0.00147, 0.00374]
53	ruletkowa	200	0.7	0.1	0.000271	0.00037	[-0.00127, -0.0025, -0.000668]
54	ruletkowa	100	0.9	0.3	0.000281	0.000319	[-0.0012, -0.00579, -0.000636]
55	ruletkowa	200	0.5	0.3	0.000292	0.000315	[0.00367, -0.00517, 0.0017]
56	ruletkowa	100	0.7	0.1	0.0003	0.000305	[0.00253, 0.00708, 0.00267]
57	ruletkowa	100	0.5	0.3	0.000354	0.000423	[-0.00268, 0.000586, 0.00676]
58	rankingowa	50	0.5	0.3	0.000371	0.000428	[-0.00172, -0.00435, 0.00171]
59	ruletkowa	200	0.9	0.1	0.00045	0.000337	[-0.00901, -0.000399, -0.000936]
60	rankingowa	100	0.5	0.1	0.000576	0.000876	[-0.0035, 0.00131, 0.00279]
61	rankingowa	50	0.9	0.1	0.000924	0.00135	[-0.000511, -0.000764, 0.00424]
62	ruletkowa	100	0.5	0.1	0.00104	0.00123	[-0.00355, 0.00776, -0.00599]
63	rankingowa	50	0.7	0.5	0.00175	0.00346	[-0.000231, -0.00202, 0.00272]
64	rankingowa	50	0.7	0.3	0.00278	0.00548	[0.00126, -0.00104, 0.000176]
65	ruletkowa	50	0.5	0.3	0.00416	0.00426	[0.0054, -0.00917, 0.00218]
66	ruletkowa	50	0.9	0.5	0.00643	0.00986	[-0.000333, -0.00249, 0.00293]
67	turniejowa	100	0.9	0.1	0.0086	0.0166	[-0.00323, -0.00139, 0.00154]
68	turniejowa	100	0.7	0.1	0.0117	0.0223	[0.00157, -0.000789, -0.00539]
69	turniejowa	50	0.5	0.3	0.0139	0.0258	[0.00258, 9.65e-06, -0.000112]
70	turniejowa	50	0.7	0.3	0.0259	0.0507	[0.000968, 0.000331, -0.00274]
71	turniejowa	100	0.5	0.1	0.0272	0.0469	[0.00382, -0.00171, -0.000121]
72	rankingowa	50	0.5	0.1	0.0467	0.0419	[0.00038, 0.000539, -0.0183]
73	turniejowa	50	0.5	0.5	0.062	0.119	[9.24e-05, 0.000204, -0.000857]
74	ruletkowa	50	0.7	0.1	0.151	0.262	[-0.00249, -0.0144, 0.00396]
75	ruletkowa	50	0.9	0.1	0.245	0.489	[-0.0145, 0.0208, -0.00149]
76	rankingowa	50	0.7	0.1	0.28	0.265	[0.0149, -0.0105, -0.0113]
77	turniejowa	50	0.7	0.1	0.337	0.41	[-0.00647, -0.0134, 0.0152]
78	ruletkowa	50	0.5	0.1	0.435	0.426	[0.0364, 0.0428, -0.0175]
79	turniejowa	50	0.5	0.1	0.692	0.86	[-0.0726, -0.16, -0.014]
80	turniejowa	50	0.9	0.1	1.05	1.44	[0.00996, 0.00682, -0.00705]

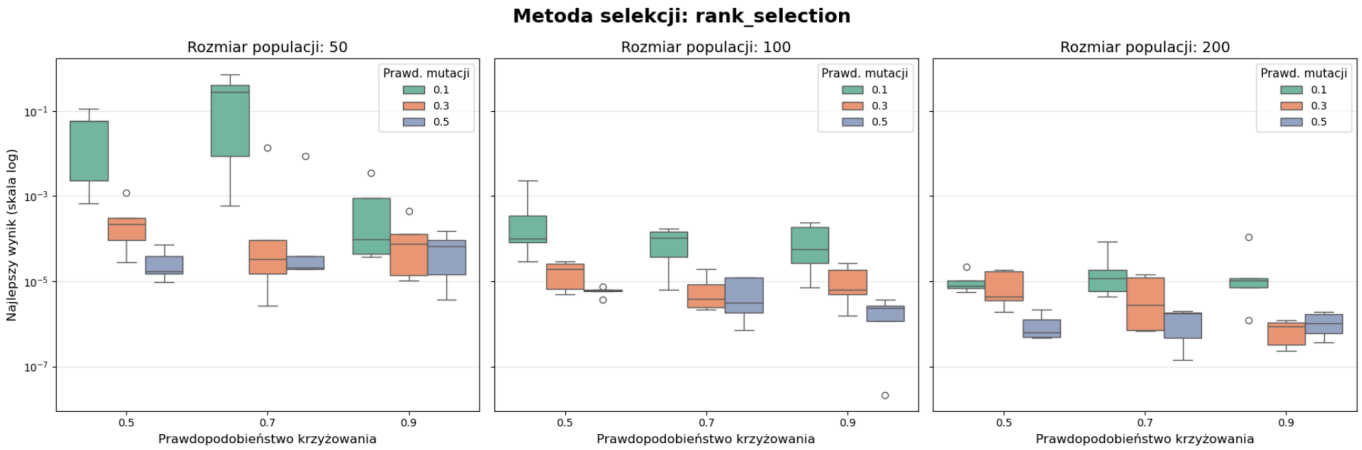
Tabela 3: Średnie wyniki funkcji celu wraz z odchyleniem standardowym dla 5 wywołań algorytmu ewolucyjnego dla funkcji Rastrigina, gdzie n oznacza rozmiar populacji, a najlepsze rozwiązanie to wartość **x** dla wywołania o najlepszej wartości funkcji celu. Wyniki są uporządkowane rosnąco względem średniego wyniki, co oznacza, że im wyżej znajduje się dany zestaw parametrów, tym lepsze wyniki uzyskiwał.

	Selekcja	n	P-stwo krzyżowania	P-stwo mutacji	Średni wynik	Odch. std.	Najlepsze rozwiązanie
0	rankingowa	200	0.7	0.3	0.00134	0.000707	[-0.000121, -0.000439, 0.000889, -0.000125, 0.00132]
1	rankingowa	200	0.9	0.3	0.2	0.398	[1.85e-05, -0.000225, 0.000746, 0.000493, -0.000521]
2	rankingowa	200	0.9	0.1	0.208	0.396	[0.000994, 0.000308, 0.00302, -0.000983, -0.00179]
3	rankingowa	200	0.9	0.5	0.398	0.487	[0.000135, 0.000374, 0.000137, -0.000665, -0.000476]
4	rankingowa	200	0.7	0.5	0.797	0.744	[-0.000583, 0.000671, -0.000651, -6.09e-05, 0.0013]
5	rankingowa	100	0.7	0.5	0.797	0.744	[0.00115, 0.00128, -0.000206, 0.00177, -0.000432]
6	ruletkowa	200	0.9	0.3	0.829	1.16	[0.00153, 0.00339, -0.00628, 0.00043, -0.00366]
7	rankingowa	200	0.5	0.3	0.996	0.629	[-8.23e-05, -0.000533, -0.00106, -0.000244, 0.000421]
8	rankingowa	100	0.9	0.5	1.2	0.975	[0.000942, -0.000733, -0.00019, 0.000339, 0.00156]
9	rankingowa	50	0.7	0.3	1.21	0.737	[0.0111, -0.00234, 0.000946, -0.00128, 0.00024]
10	ruletkowa	200	0.5	0.3	1.26	0.372	[0.00389, 0.00494, 0.997, -0.00143, 0.00258]
11	rankingowa	200	0.5	0.5	1.39	1.35	[0.0001, -0.00153, 0.000357, 0.000564, -0.000129]
12	turniejowa	200	0.9	0.5	1.39	1.19	[-0.000269, -0.000275, 0.000203, 8.99e-05, -0.00052]
13	rankingowa	100	0.5	0.5	1.4	1.35	[-0.00236, -0.00133, -0.00116, -0.00149, -0.000124]
14	ruletkowa	200	0.7	0.1	1.52	1.23	[-0.000737, 0.00636, -0.0139, 0.00834, -0.00322]
15	turniejowa	200	0.9	0.3	1.59	0.487	[0.996, -0.00062, 0.00043, -0.00105, -9.29e-05]
16	rankingowa	200	0.7	0.1	1.6	1.02	[-4.57e-05, 0.00214, 0.00148, -0.000555, -0.00308]
17	turniejowa	200	0.7	0.1	1.6	1.01	[-0.00043, 0.00713, -0.00134, -0.00212, 0.000161]
18	turniejowa	200	0.9	0.1	1.6	1.02	[-0.000252, -0.00137, 0.00333, 0.000924, -0.000472]
19	rankingowa	100	0.7	0.1	1.62	1.6	[0.00429, -0.00919, -0.00551, 0.000458, 0.00388]
20	ruletkowa	200	0.9	0.5	1.63	0.799	[0.000232, -0.00376, 0.987, -0.0015, -0.0039]
21	turniejowa	200	0.7	0.3	1.79	1.32	[-0.000185, -0.00174, -0.000711, 0.000267, -0.00109]
22	rankingowa	100	0.9	0.3	1.79	0.745	[-0.000557, 0.000879, 0.00142, -0.00116, 0.996]
23	rankingowa	200	0.5	0.1	1.8	0.966	[0.000323, -0.0101, -0.00118, 0.00749, -0.00337]
24	rankingowa	100	0.7	0.3	1.8	1.16	[-0.000895, 0.000543, -3.85e-06, 0.00453, 0.00154]
25	rankingowa	100	0.9	0.1	1.81	1.71	[0.00422, -0.00667, -0.00188, -0.00462, 0.00162]
26	ruletkowa	200	0.9	0.1	1.87	1.33	[0.00156, -0.0105, -0.0111, 0.0158, 0.00401]
27	ruletkowa	200	0.7	0.5	2.03	1.42	[0.00398, -0.00513, -0.00608, -0.00164, -0.00285]
28	turniejowa	200	0.7	0.5	2.19	1.16	[-0.000488, -0.995, -0.000637, 0.000186, -0.000684]
29	rankingowa	50	0.7	0.1	2.47	1.36	[-0.00231, 0.0114, -0.0174, -0.00307, -0.992]
30	rankingowa	100	0.5	0.3	2.59	2.14	[0.00332, -0.00101, -0.00282, 0.00056, 0.00316]
31	turniejowa	200	0.5	0.1	2.61	2.41	[-0.00122, 0.00361, 0.00225, -0.0019, 0.000171]
32	rankingowa	100	0.5	0.1	2.82	1.45	[-0.0007, 0.00228, -0.00968, -7.6e-05, 0.981]
33	turniejowa	200	0.5	0.3	2.99	1.09	[-0.000282, -7.29e-05, 0.996, -0.000133, 0.994]
34	rankingowa	50	0.9	0.3	3.0	1.67	[-0.000282, 0.00669, 0.99, -0.00498, 0.00152]
35	ruletkowa	100	0.7	0.5	3.02	1.08	[0.000749, -0.992, 0.0059, 0.992, 0.00679]
36	ruletkowa	200	0.5	0.1	3.06	1.74	[-1.0, -0.0249, -0.0179, 0.0123, -0.0015]
37	turniejowa	200	0.5	0.5	3.18	1.16	[6.28e-05, -0.000384, -0.995, 0.995, -9.06e-05]
38	turniejowa	100	0.9	0.5	3.19	2.03	[-0.000883, -0.0014, 0.995, -4.28e-05, 0.00124]
39	turniejowa	100	0.9	0.3	3.19	1.46	[0.000843, -0.995, 9.63e-06, 0.995, 0.000566]
40	ruletkowa	100	0.9	0.5	3.21	1.46	[0.994, -0.000619, -0.00525, 0.993, -0.00152]
41	ruletkowa	200	0.5	0.5	3.22	2.02	[-0.00855, 1.96e-05, 0.00259, -0.999, 0.00181]
42	rankingowa	50	0.9	0.5	3.39	1.85	[-0.000439, 0.000458, -0.000648, 0.996, 0.000944]
43	turniejowa	100	0.5	0.1	3.42	1.6	[-0.018, -0.00181, -0.00604, -0.00834, 0.999]
44	ruletkowa	100	0.9	0.3	3.44	1.93	[1.0, 0.00655, -0.00537, 0.0149, 0.00735]
45	turniejowa	100	0.7	0.1	3.63	1.61	[0.998, -0.0109, 0.00545, -0.00971, -0.996]
46	ruletkowa	50	0.7	0.5	3.64	2.07	[-0.00555, -1.0, 0.00449, 0.00495, -0.00202]
47	turniejowa	100	0.7	0.5	3.78	1.46	[-0.996, -0.00149, -0.994, -0.000912, 9.48e-05]
48	ruletkowa	200	0.7	0.3	3.82	1.72	[-0.000724, 1.0, 0.00476, 0.00576, -0.00439]
49	rankingowa	50	0.9	0.1	3.88	0.731	[-0.00254, 1.0, 0.00477, -1.0, 0.999]
50	ruletkowa	100	0.5	0.5	4.01	2.08	[-0.998, 0.00461, -0.994, 0.0038, 0.00112]
51	ruletkowa	50	0.9	0.3	4.02	2.43	[0.00485, -0.0113, 0.00792, 0.99, 0.00614]
52	turniejowa	100	0.5	0.5	4.18	1.71	[-0.00126, -0.000892, 0.995, 0.00151, -0.996]
53	turniejowa	50	0.9	0.5	4.19	1.16	[0.000143, -0.996, 0.994, -0.989, -0.00085]
54	turniejowa	100	0.7	0.3	4.19	1.6	[0.997, -0.994, 0.00202, 0.0007, -0.993]
55	rankingowa	50	0.5	0.3	4.2	3.11	[-0.00193, -0.00326, -0.997, -0.00121, 8.46e-06]
56	ruletkowa	100	0.5	0.3	4.22	2.87	[-0.00453, -0.996, 0.00572, -0.00146, 0.995]
57	ruletkowa	50	0.9	0.1	4.25	1.79	[0.00213, -0.996, 1.0, -0.0111, 0.0051]

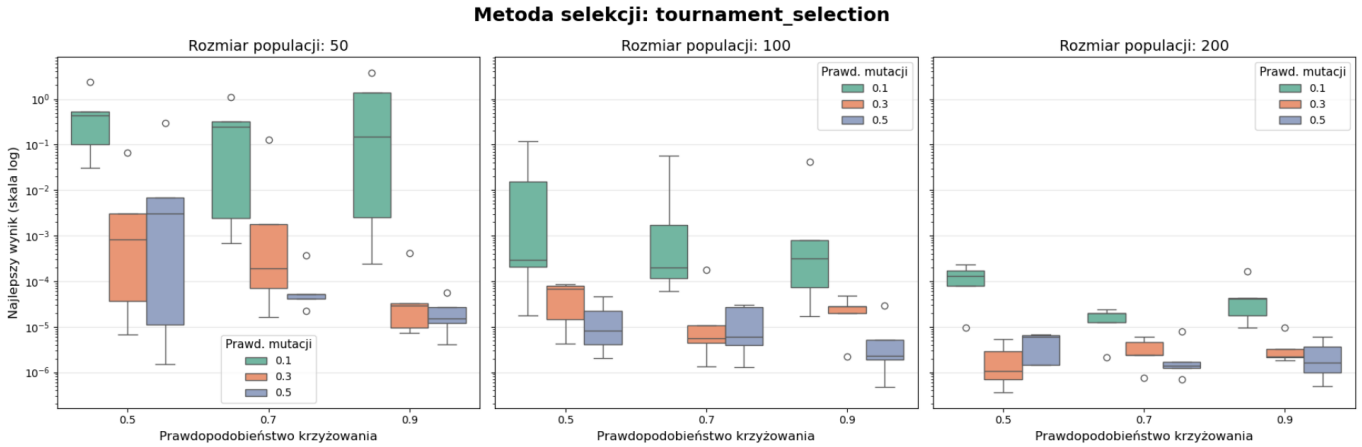
Ciąg dalszy na następnej stronie...

Tabela 3 - ciąg dalszy z poprzedniej strony

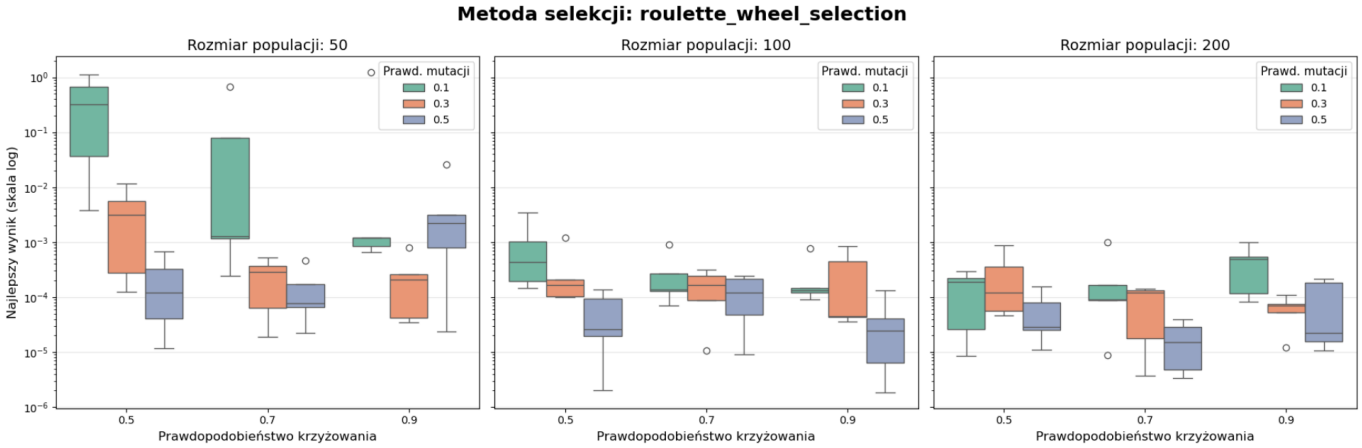
	Selekcja	n	P-stwo krzyżowania	P-stwo mutacji	Średni wynik	Odch. std.	Najlepsze rozwiązanie
58	turniejowa	50	0.9	0.1	4.32	1.09	[0.0231, 0.992, -0.002, -0.997, -0.991]
59	rankingowa	50	0.5	0.1	4.47	1.81	[-0.0182, 0.0051, -0.985, -1.0, 0.00914]
60	ruletkowa	50	0.9	0.5	4.6	4.06	[0.00262, -0.00329, -0.00172, 0.00911, -0.00264]
61	ruletkowa	100	0.7	0.3	4.62	2.55	[0.00226, -0.0162, 0.0134, 0.994, -0.00258]
62	ruletkowa	100	0.9	0.1	4.91	2.23	[-0.0073, -0.00339, 0.992, -0.000721, -0.977]
63	turniejowa	100	0.9	0.1	5.05	2.07	[-0.0138, -1.0, -0.000859, -1.0, -0.000101]
64	ruletkowa	100	0.7	0.1	5.26	2.05	[-0.987, -0.999, -0.005, -0.00845, -0.0053]
65	rankingowa	50	0.7	0.5	5.38	2.32	[-0.993, -0.995, -0.000328, -0.00038, 0.996]
66	turniejowa	50	0.7	0.1	6.52	2.7	[0.992, -0.996, -0.991, -0.992, -0.00355]
67	turniejowa	100	0.5	0.3	6.57	3.59	[-0.000795, -0.995, 0.00155, 0.00129, -0.00257]
68	turniejowa	50	0.7	0.3	6.59	2.92	[0.998, -0.996, -0.00499, 0.984, -0.0101]
69	ruletkowa	50	0.5	0.3	6.63	6.03	[-0.984, 0.000926, 0.00321, -0.00425, -0.00602]
70	turniejowa	50	0.7	0.5	6.97	2.74	[-0.000664, 0.994, -0.995, -0.995, 0.995]
71	rankingowa	50	0.5	0.5	7.37	3.42	[0.00259, 0.996, -0.00184, 0.00668, -0.995]
72	turniejowa	50	0.9	0.3	7.38	1.73	[-0.998, -0.994, -1.99, 0.00108, -0.00655]
73	ruletkowa	50	0.5	0.1	7.47	4.48	[-0.00742, 0.00472, -0.00234, 0.0113, 0.0186]
74	turniejowa	50	0.5	0.5	7.57	2.15	[-0.997, 0.995, -0.995, 0.995, 0.995]
75	ruletkowa	50	0.5	0.5	7.79	1.31	[0.0018, -1.98, -0.00286, -1.01, 0.992]
76	ruletkowa	50	0.7	0.1	7.8	2.78	[0.997, -0.993, -0.0067, 1.0, 0.00654]
77	ruletkowa	100	0.5	0.1	8.03	3.0	[1.0, -0.974, -0.993, 0.00149, -1.0]
78	turniejowa	50	0.5	0.1	8.72	2.73	[-0.0125, -1.99, -0.0097, -0.000924, 0.981]
79	turniejowa	50	0.5	0.3	9.18	6.56	[0.00174, -0.993, -0.00379, -0.996, -0.00433]
80	ruletkowa	50	0.7	0.3	9.21	2.77	[1.98, -0.00105, -0.994, -0.000853, -0.997]



(a) Selekcja rankingowa

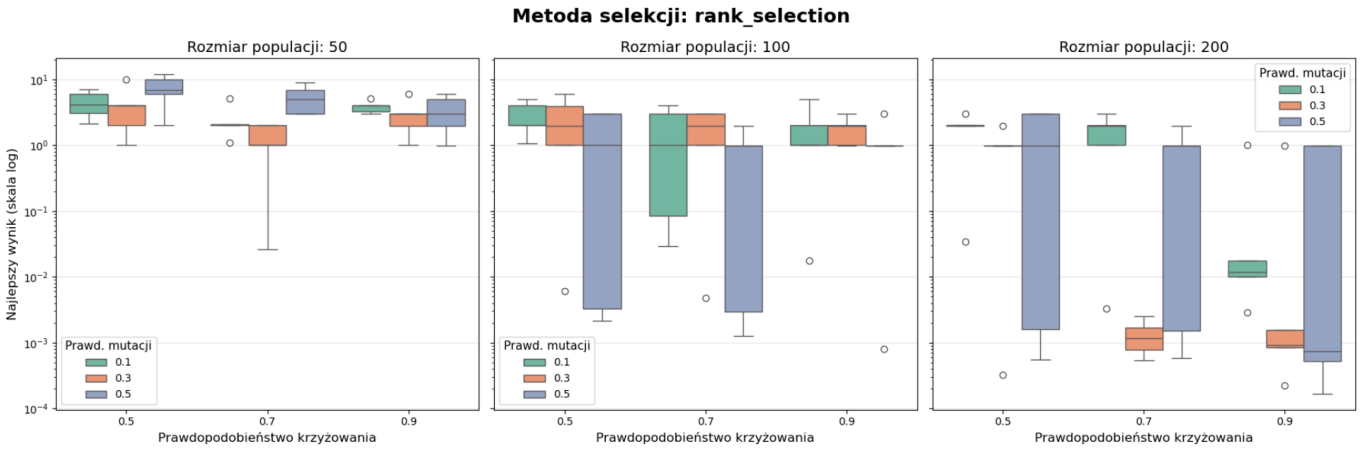


(b) Selekcja turniejowa

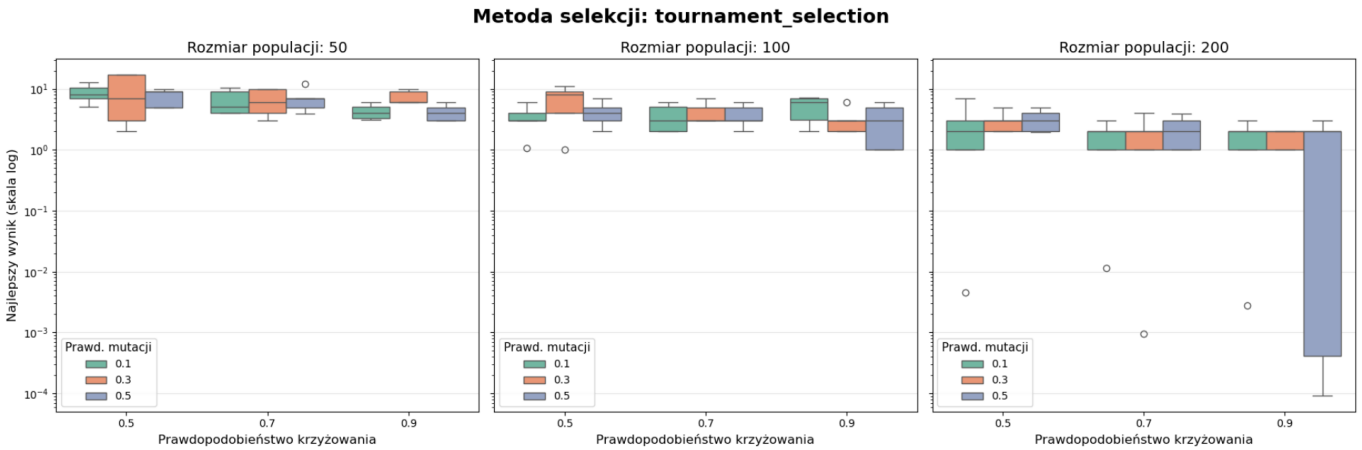


(c) Selekcja ruletkowa

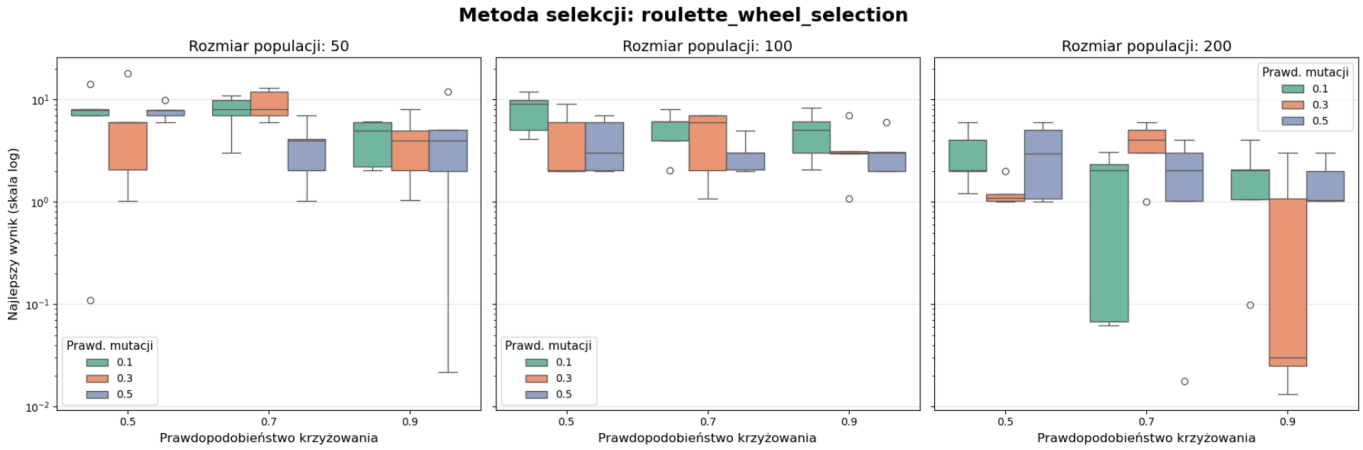
Rysunek 1: Wyniki algorytmów ewolucyjnych dla funkcji trójwymiarowej o różnych parametrach i algorytmach selekcji.



(a) Selekcja rankingowa

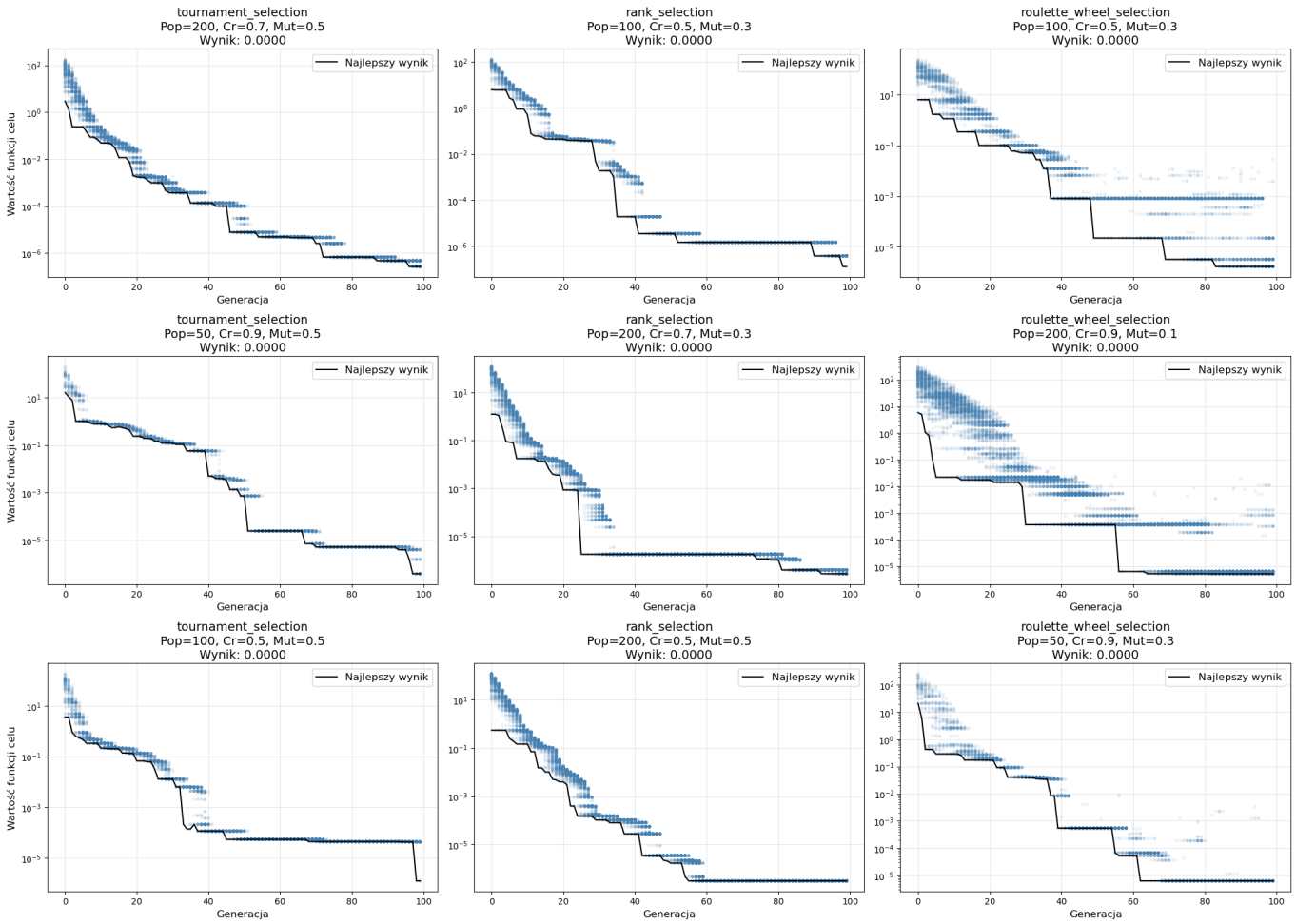


(b) Selekcja turniejowa

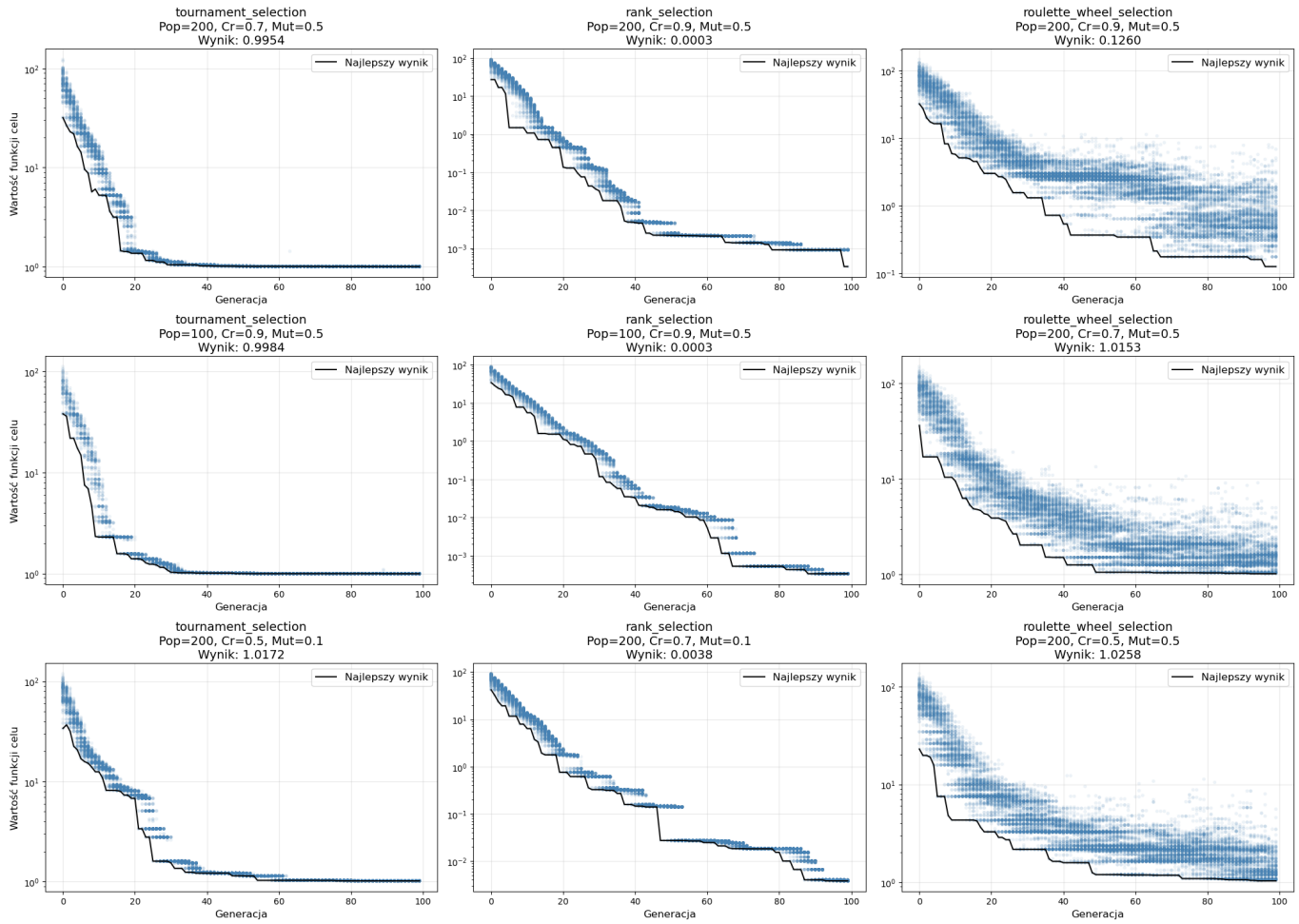


(c) Selekcja ruletkowa

Rysunek 2: Wyniki algorytmów ewolucyjnych dla funkcji Rastrigina o różnych parametrach i algorytmach selekcji.



Rysunek 3: Przebieg generacji dla trzech najlepszych modeli dla każdego z algorytmów selekcji dla funkcji trójwymiarowej. Pop oznacza rozmiar populacji, Cr prawdopodobieństwo krzyżowania, a Mut prawdopodobieństwo mutacji. Niebieskie kropki oznaczają osobników danej generacji, a czarną linią oznaczony jest najlepszy wynik danej generacji.



Rysunek 4: Przebieg generacji dla trzech najlepszych modeli dla każdego z algorytmów selekcji dla funkcji Rastrigina. Pop oznacza rozmiar populacji, Cr prawdopodobieństwo krzyżowania, a Mut prawdopodobieństwo mutacji. Niebieskie kropki oznaczają osobników danej generacji, a czarną linią oznaczony jest najlepszy wynik danej generacji.

Analizując wyniki, można wywnioskować, że zwiększanie rozmiaru populacji w każdym przypadku poprawia wyniki, ponieważ pozwala na lepsze przeszukanie przestrzeni. Wyższe prawdopodobieństwo mutacji powoduje większą losowość wyników, co widać po większym odchyleniu standardowym. Większe prawdopodobieństwo krzyżowania nieznacznie poprawia wyniki i powoduje szybsze dążenie do minimów lokalnych. Na podanych funkcjach najlepiej sprawdziła się selekcja rankingowa. Na funkcji Rastrigina dawała większe odchylenie standardowe wyników, ale dzięki temu zdarzały się wywołania, na których osiągała najlepsze wyniki. Gorsze wyniki otrzymywał algorytm z selekcją turniejową i ruletkową. Na funkcji trójwymiarowej lepsze wyniki dawała selekcja turniejowa, na funkcji Rastrigina ruletkowa, natomiast były to nieznaczne różnice i obie selekcje dawały zadowalające wyniki. Widoczne było również, że selekcja rankingowa i turniejowa sprawiały, że cała populacja znajdowała się w pobliżu najlepszej wartości, w przeciwieństwie do ruletki, gdzie zakres wyników osobników populacji był większy. Wprowadziło to różnorodność genetyczną, która pozwalała algorytmowi na szerszą eksplorację przestrzeni rozwiązań i zmniejszała ryzyko utknięcia w minimum lokalnym.

Wypełnianie koła prostokątami

Na tym etapie algorytm ewolucyjny został użyty do rozwiązania problemu znanego w literaturze jako problem cutting stock. Dane jest koło o promieniu r oraz zbiór dostępnych prostokątów opisanych trzema wartościami: wysokości, szerokości i wartości. Celem jest ułożenie prostokątów w kole tak, aby zmaksymalizować sumę ich wartości, przy zachowaniu następujących warunków: boki wszystkich prostokątów muszą być równoległe do osi układu, wnętrza prostokątów nie mogą mieć części wspólnej, każdy typ prostokąta można wstawić dowolnie wiele razy.

Aby rozwiązać ten problem dla danego promienia okręgu, stworzony został najpierw zbiór wszystkich dostępnych prostokątów wraz z ich obróconymi wariantami dla danego zbioru. Osobnikiem w algorytmie ewolucyjnym jest kolejność, w jakiej prostokąty są ustawiane w okręgu. Każdy z nich jest po kolei wstawiany do okręgu przez szukanie pierwszego wolnego miejsca, idąc od punktu (0,0) obszaru. Co ważne, algorytm nie przeszukuje każdego możliwego miejsca dla prostokąta, a jedynie co 20. piksel, aby zoptymalizować czas wykonywania przeszukiwań. Dla wybranej pozycji są sprawdzane rogi prostokąta - gdy wychodzą poza granice okręgu lub nachodzą na inny prostokąt, to obszar jest przeszukiwany dalej. W przeciwnym wypadku sprawdzane są pozostałe współrzędne prostokąta i odpowiednio

jest on wstawiany lub szukane jest dla niego inne miejsce. W przypadku wyczerpania wszystkich możliwych pozycji algorytm przechodzi do próby dopasowania kolejnego elementu z sekwencji. Długość osobnika, czyli maksymalna liczba prostokątów, dla których następuje próba ustawienia, jest ustalana przy inicjacji algorytmu. Dla każdego ze zbiorów wartość ta była wybierana eksperymentalnie.

Osobniki poddawane są działaniu algorytmu ewolucyjnego, a ich ewaluacja polega na symulacji ułożenia prostokątów w okręgu i zsumowaniu ich wartości. Aby móc korzystać z niezmienionego algorytmu z pierwszego etapu, zwracana jest wartość przeciwna do uzyskanej, aby móc szukać minimum.

Jako algorytm krzyżowania jest używane zdefiniowane wcześniej krzyżowanie jednopunktowe. Do mutacji jest używany algorytm mutacji dyskretnej, który 10% z prostokątów z kolejności zmienia rodzaj na inny, wylosowany ze zbioru dostępnych typów. Jako algorytm selekcji wybrano selekcję rankingową, która dawała zwykle najlepsze wyniki w etapie pierwszym. Pozostałe parametry użyte w modelach dla konkretnych wartości promienia są umieszczone w tabeli 4.

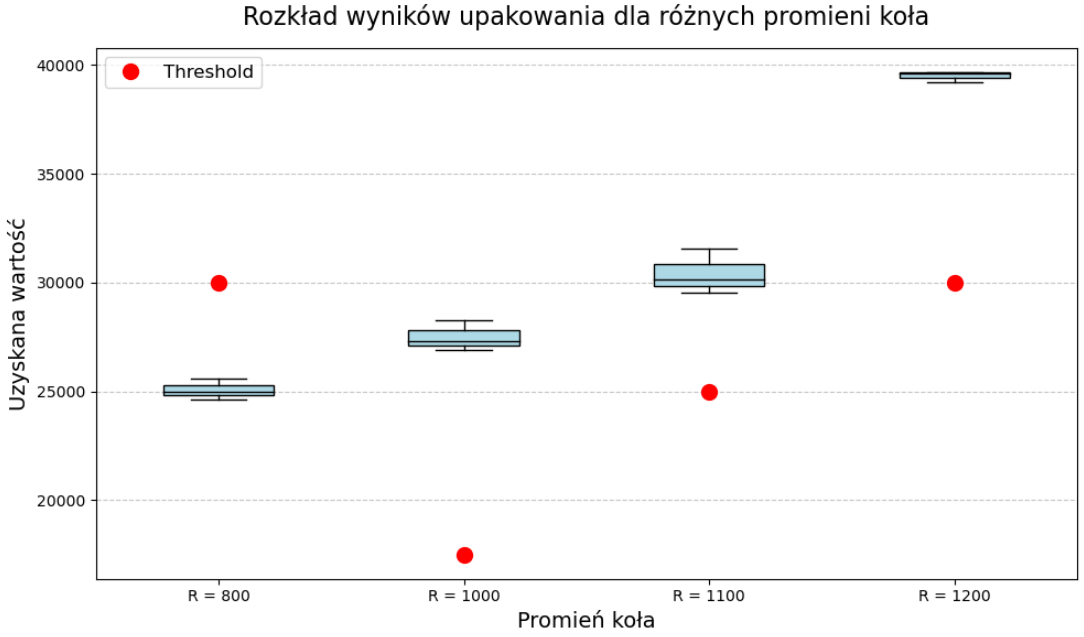
Tabela 4: Parametry używane w modelach dla konkretnych wartości promienia okręgu wraz z oczekiwanym wynikiem.

Promień	Rozmiar populacji	Długość osobnika	P-stwo krzyżowania	P-stwo mutacji	Liczba generacji
800	30	100	0.9	0.5	30
850		200			10
1000		120			
1100		120			
1200		160			

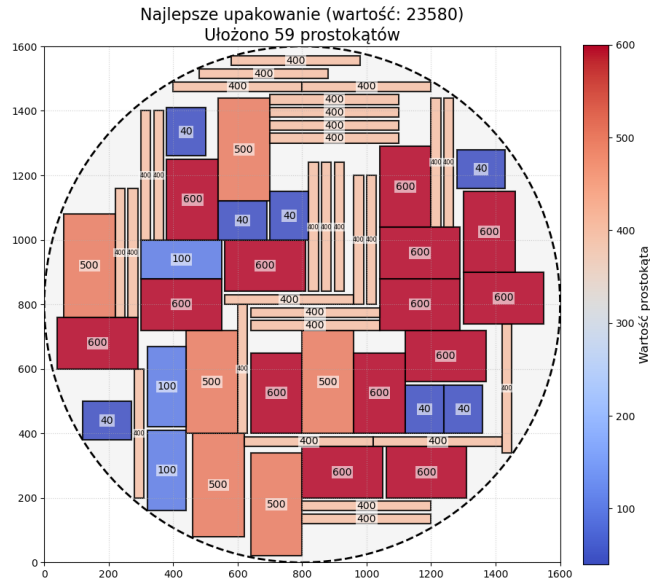
Wyniki uzyskane po 3 wywołaniach każdego z modeli są umieszczone w tabeli 5 oraz na wykresie 5. Zobrazowanie jednego z uzyskanych ustawień prostokątów dla każdego z promieni jest widoczne na wykresach 6, natomiast przebieg generacji jest widoczny na wykresach 7.

Tabela 5: Średnie sumy wartości prostokątów umieszczonych w okręgach o danym promieniu wraz z odchyleniem standardowym, najmniejszym i największym wynikiem uzyskanym po 3 wywołaniach. Threshold jest oczekiwanym wynikiem, który należało otrzymać dla danego promienia.

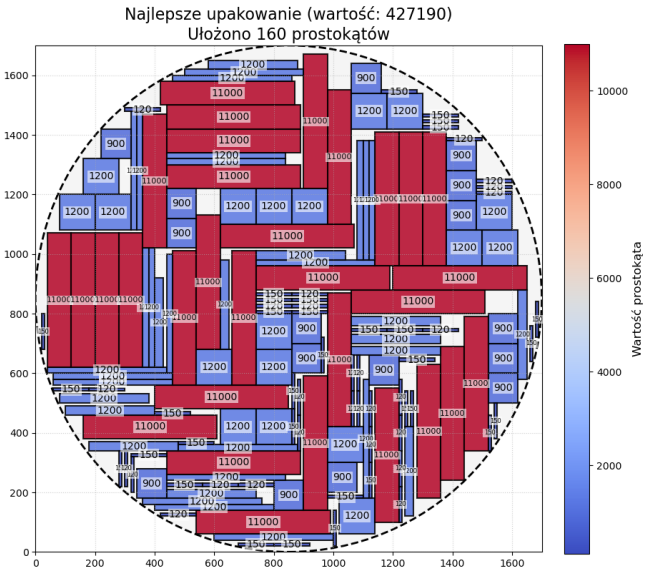
Promień	Średni wynik	Odch. std.	Minimum	Maksimum	Threshold
800	25066.67	388.62	24640	25580	30000
850	440120.00	5435.02	432460	444500	-
1000	27500.00	588.78	26900	28300	17500
1100	30420.00	860.85	29520	31580	25000
1200	39506.67	204.18	39220	39680	30000



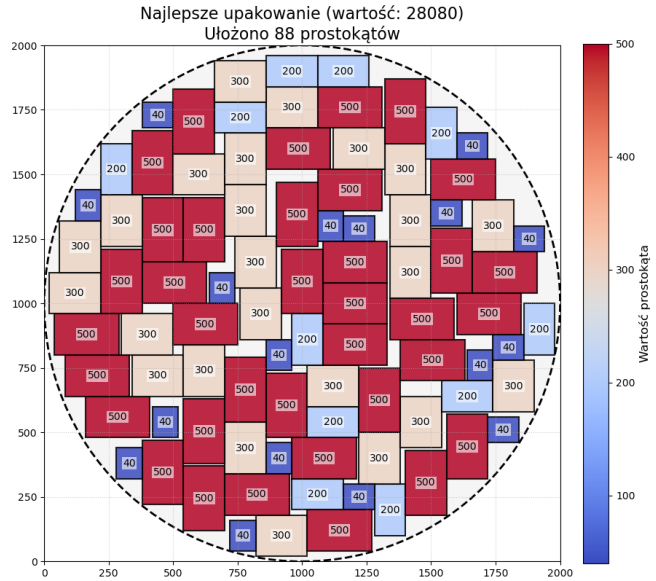
Rysunek 5: Wyniki uzyskane dla różnych promieni okręgu w 3 wywołaniach algorytmu ewolucyjnego (wyniki dla promienia równego 850 nie są uwzględnione dla poprawy czytelności). Czerwoną kropką są zaznaczone oczekiwane wyniki.



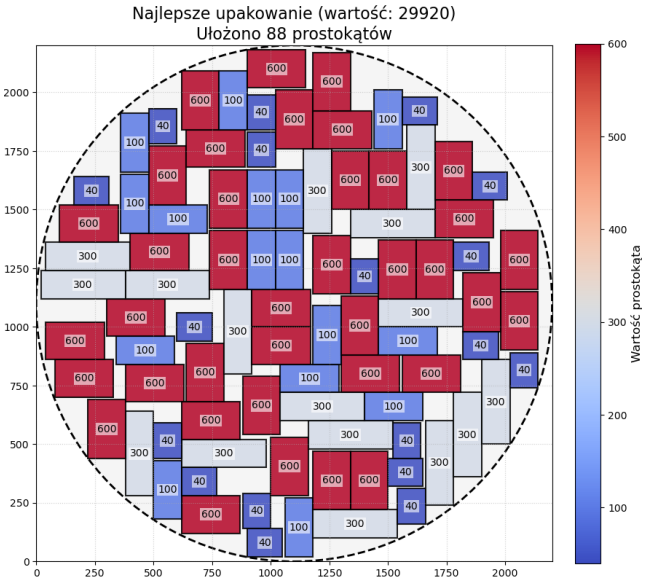
(a) $R = 800$, Threshold $= 30000$



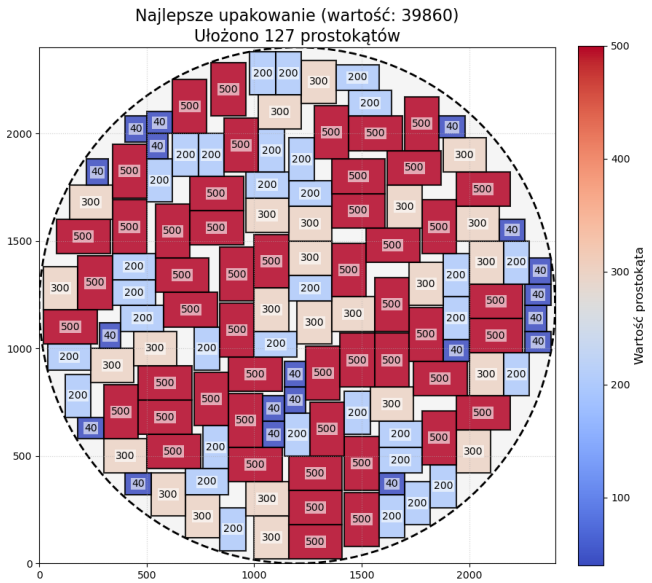
(b) $R = 850$



(c) $R = 1000$, Threshold $= 17500$

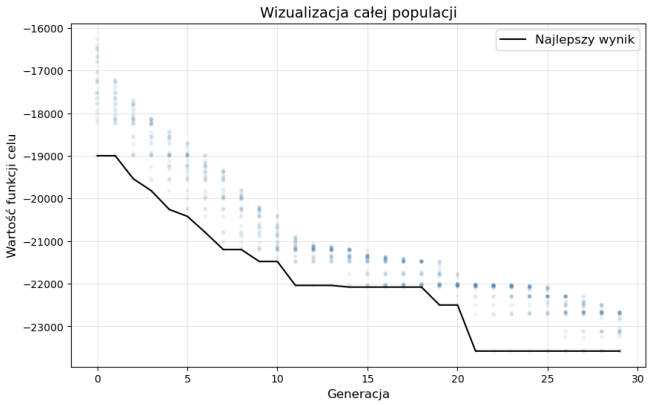


(d) $R = 1100$, Threshold $= 25000$

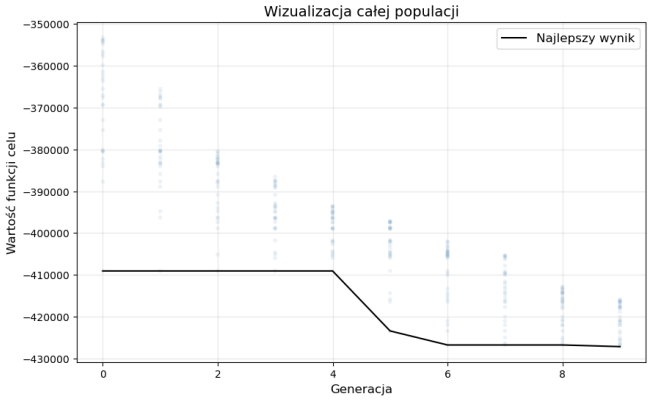


(e) $R = 1200$, Threshold $= 30000$

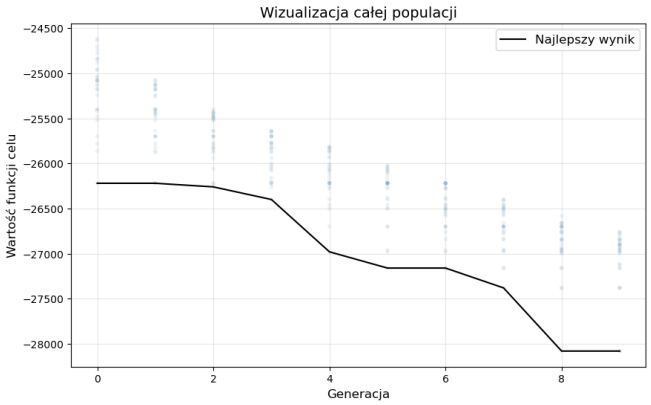
Rysunek 6: Wyniki upakowania prostokątów przez algorytm ewolucyjny dla różnych wartości promienia okręgu (R) i oczekiwanej wartości wyniku (Threshold).



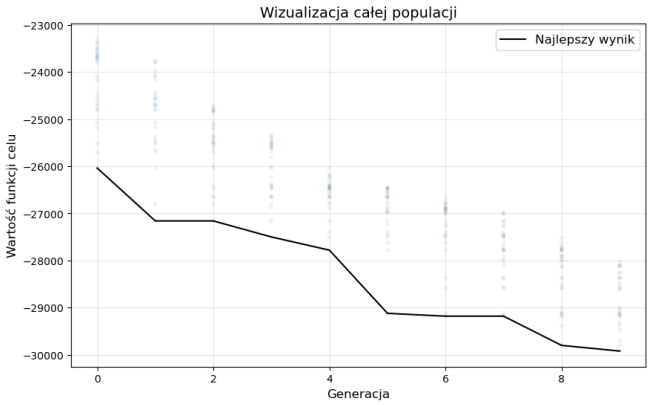
(a) $R = 800$, Threshold = 30000



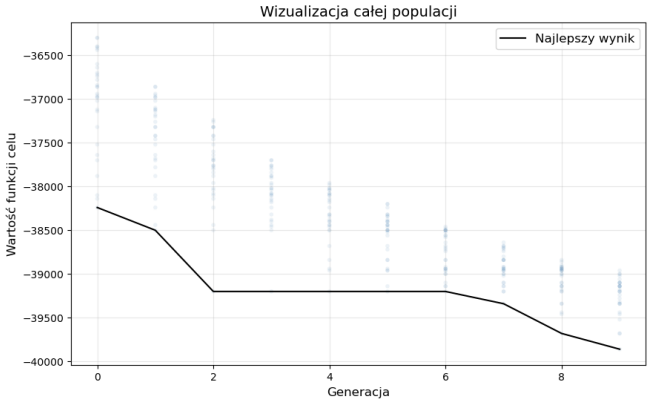
(b) $R = 850$



(c) $R = 1000$, Threshold = 17500



(d) $R = 1100$, Threshold = 25000



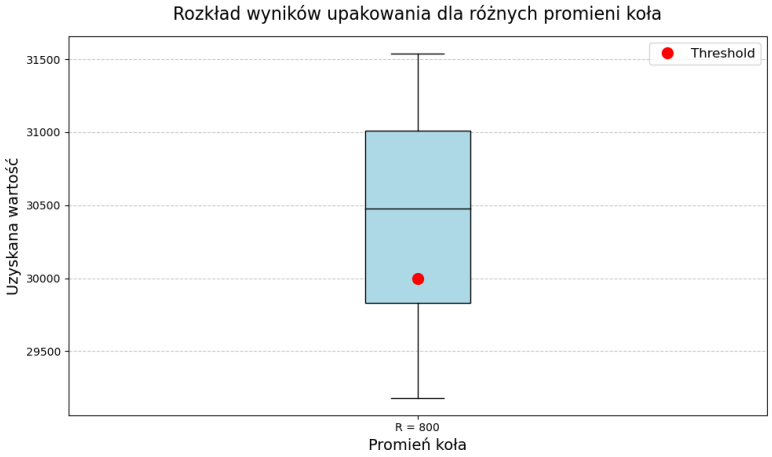
(e) $R = 1200$, Threshold = 30000

Rysunek 7: Przebieg generacji dla różnych wartości promienia okręgu (R) i oczekiwanej wartości wyniku (Threshold). Ponieważ algorytm korzystał z minimalizacji, wartość funkcji celu jest wartością przeciwną do rzeczywistego wyniku upakowania.

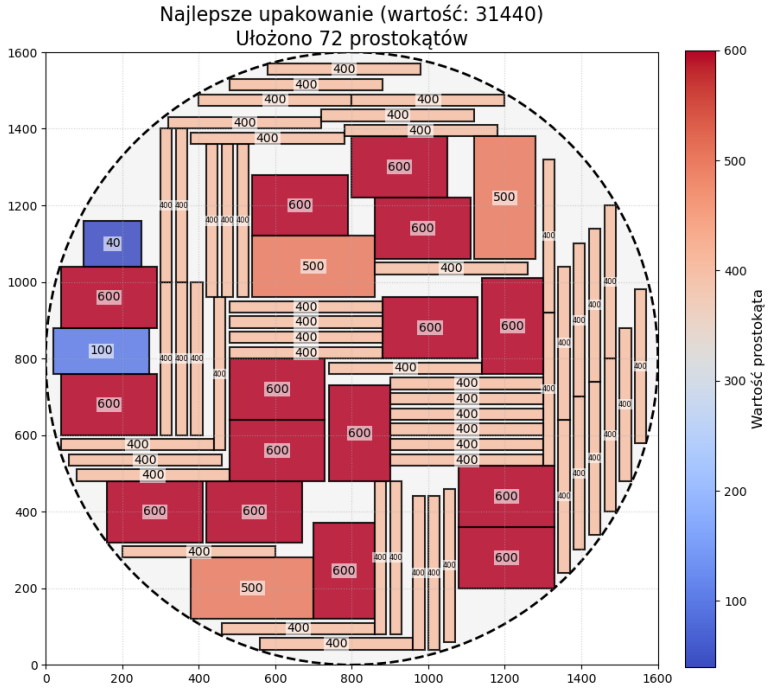
Aby poprawić wyniki dla promienia równego 800, wprowadzono zmianę w inicjalizacji zbioru prostokątów. Zamiast ustawiać je całkowicie losowo, prawdopodobieństwo wylosowania prostokąta zależało od stosunku wartości prostokąta do jego pola. Dzięki temu algorytm zaczynał od ustawiania prostokątów, które miały największe wartości przy najmniejszym polu. Po tej zmianie uzyskano wyniki widoczne w tabeli 6 i na wykresie 8 oraz upakowanie widoczne na wykresie 9. Analizując przebieg generacji 10 można zauważyć, że wprowadzając tę zmianę, już inicjalny wynik upakowania jest lepszy, przez co poprawa w kolejnych generacjach nie jest znacząca.

Tabela 6: Średnie sumy wartości prostokątów umieszczonych w okręgu o promieniu 800 wraz z odchyleniem standardowym, najmniejszym i największym wynikiem uzyskanym po 3 wywołaniach. Threshold jest oczekiwanym wynikiem, który należało otrzymać dla tego promienia.

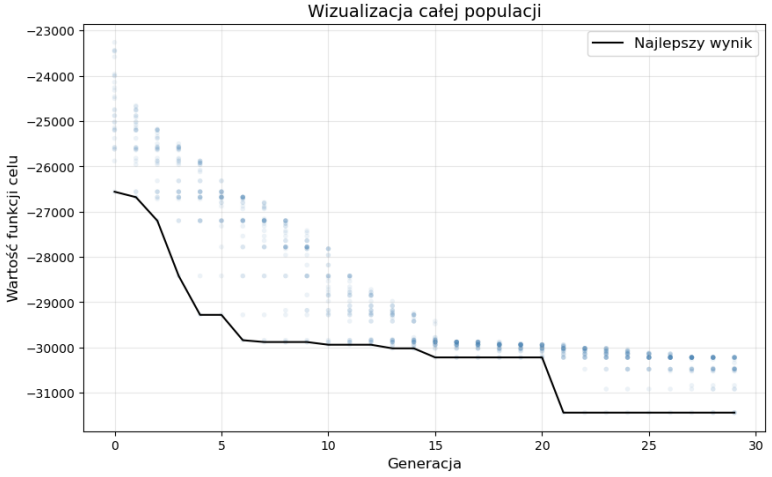
Promień	Średni wynik	Odch. std.	Minimum	Maksimum	Threshold
800	30400.0	965.13	29180	31540	30000



Rysunek 8: Wyniki uzyskane dla okręgu o promieniu 800 po zmianie inicjalizacji zbioru w 3 wywołaniach algorytmu ewolucyjnego. Czerwoną kropką są zaznaczone oczekiwane wyniki.



Rysunek 9: Wyniki upakowania prostokątów przez algorytm ewolucyjny dla promienia 800 po zmianie inicjalizacji zbioru.



Rysunek 10: Przebieg generacji dla promienia 800 po zmianie inicjalizacji zbioru.

Wizualnie można ocenić, że wykorzystanie takiego algorytmu ewolucyjnego rozwiązuje z powodzeniem problem cutting stock. Można również zauważyć, że poza promieniem 850, dla którego dane były prostokąty o znacząco wyższych wartościach, wyniki rosną wraz z promieniem - im promień okręgu jest większy, tym łatwiej jest programowi znaleźć najlepsze możliwe ułożenie prostokątów.

Optymalizacja wag w sieci MLP z użyciem algorytmu ewolucyjnego

Na tym etapie zaimplementowane zostało proste uczenie MLP z użyciem algorytmu ewolucyjnego. Pojedynczego osobnika reprezentuje wektor wag i obciążeń (biasów), który jest inicjalizowany losowymi wartościami z rozkładu jednostajnego z zakresu (-1, 1). Funkcja oceny wylicza wartość funkcji straty przekazanego modelu MLP (Crossentropy dla klasyfikacji i MSE dla regresji). Wyniki takiego algorytmu ewolucyjnego są porównywane z wynikami uczenia tej samej sieci neuronowej algorytmem propagacji wstecznej błędu. Cała architektura MLP pochodzi z ostatecznej wersji otrzymanej po zadaniu NN6. Wszystkie testy są wykonywane na zbiorach iris, auto-mpg i multimodal-large.

Przed wejściem do modeli wszystkie zbiory danych poddano procesowi normalizacji. Model wielowarstwowego perceptronu (MLP) w obu metodach korzystał ze współczynnika uczenia równego 0.01, optymalizacji metodą momentu, rozmiaru wsadu (batch size) równego 32, inicjalizacji wag metodą He oraz tangensa hiperbolicznego jako funkcji aktywacji w warstwach ukrytych. Dodatkowo, w przypadku problemu klasyfikacji zastosowano funkcję aktywacji Softmax na warstwie wyjściowej. Algorytm ewolucyjny natomiast zawsze korzystał z krzyżowania jednopunktowego z prawdopodobieństwem krzyżowania 0.8 i typowej mutacji gaussowskiej. Wyjątkiem są dane multimodal-large, gdzie użyta została mutacja gaussowska z przesunięciem losowanym z rozkładu normalnego o odchyleniu standardowym równym 0.2 (w przeciwieństwie do 0.1). Pozostałe użyte parametry są umieszczone w tabeli 7.

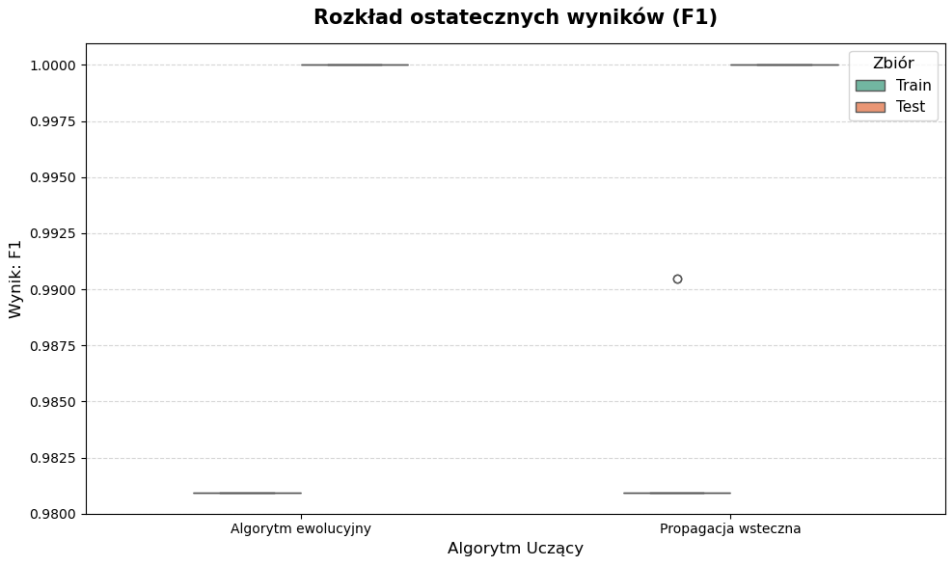
Tabela 7: Zestawienie zmiennych hiperparametrów modeli i algorytmu ewolucyjnego dla poszczególnych zbiorów danych, gdzie k przy selekcji turniejowej oznacza liczbę osobników w pojedynczym turnieju.

Zbiór danych	Architektura	Generacje / Epoki	Rozmiar populacji	Selekcja	P-stwo mutacji	Funkcja straty
iris	4-16-3	500	50	Rankingowa	0.3	Crossentropy
auto-mpg	7-16-1	500	100	Rankingowa	0.3	MSE
multimodal-large	1-16-16-1	3000	100	Turniejowa ($k = 5$)	0.5	MSE

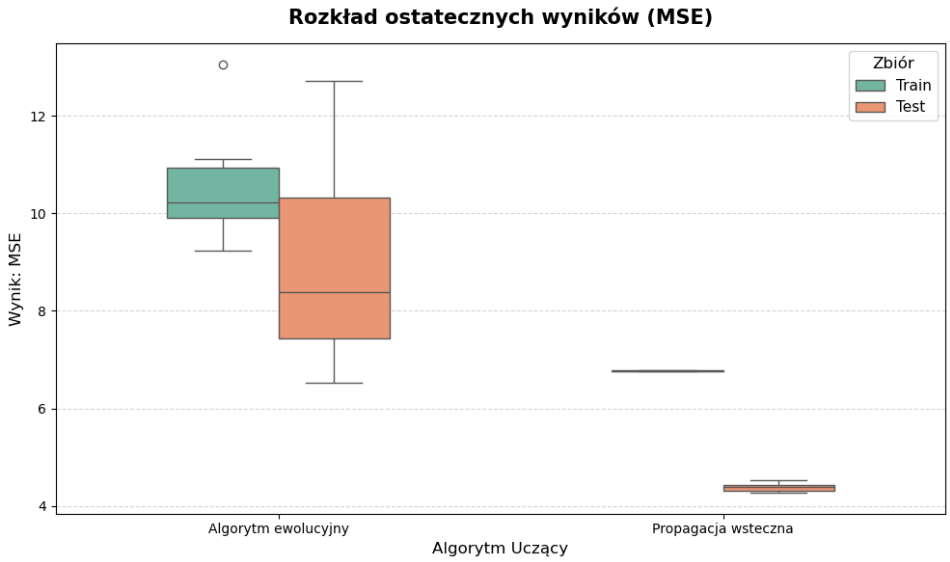
Porównanie działania modeli zostało wykonane 10 razy dla zbiorów iris i auto-mpg oraz 3 razy dla zbioru multimodal-large. Porównanie wyników jest widoczne w tabeli 8 oraz na wykresach 11. Krzywe uczenia oraz porównanie wyników dla jednego z wywołań widoczne są na wykresach 12.

Tabela 8: Porównanie średnich wyników wraz z odchyleniem standardowym dla kolejnych zbiorów danych dla algorytmu ewolucyjnego i propagacji wstecznej.

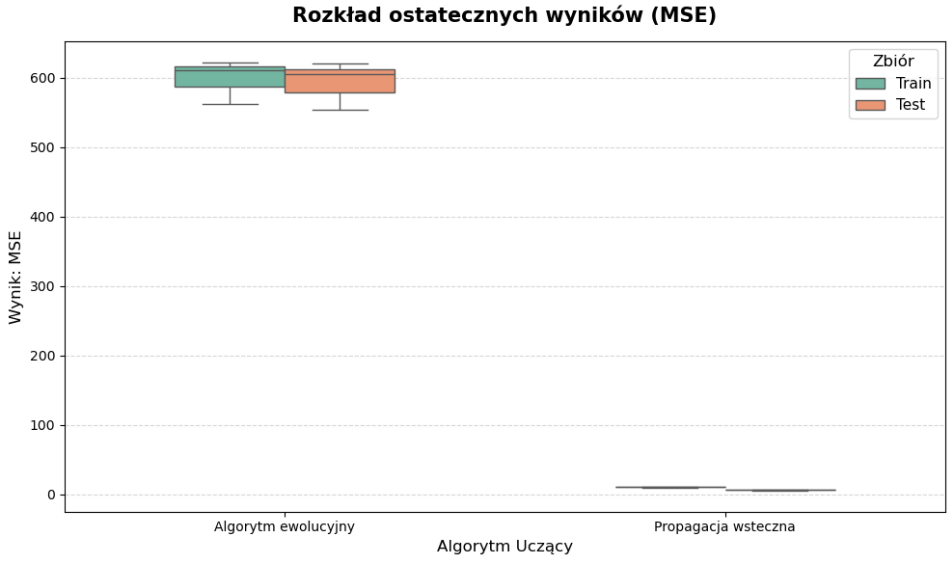
Dane	Metoda	Dane	Średni wynik	Odch. std.	Liczona wartość
iris	Algorytm ewolucyjny	Treningowe	0.981	0.000	F1
		Testowe	1.000	0.000	
	Propagacja wsteczna	Treningowe	0.983	0.004	
		Testowe	1.000	0.000	
auto-mpg	Algorytm ewolucyjny	Treningowe	10.469	1.037	MSE
		Testowe	8.898	1.860	
	Propagacja wsteczna	Treningowe	6.772	0.010	
		Testowe	4.382	0.080	
multimodal-large	Algorytm ewolucyjny	Treningowe	598.391	25.447	MSE
		Testowe	593.282	28.513	
	Propagacja wsteczna	Treningowe	10.291	0.393	
		Testowe	6.592	0.622	



(a) Zbiór iris

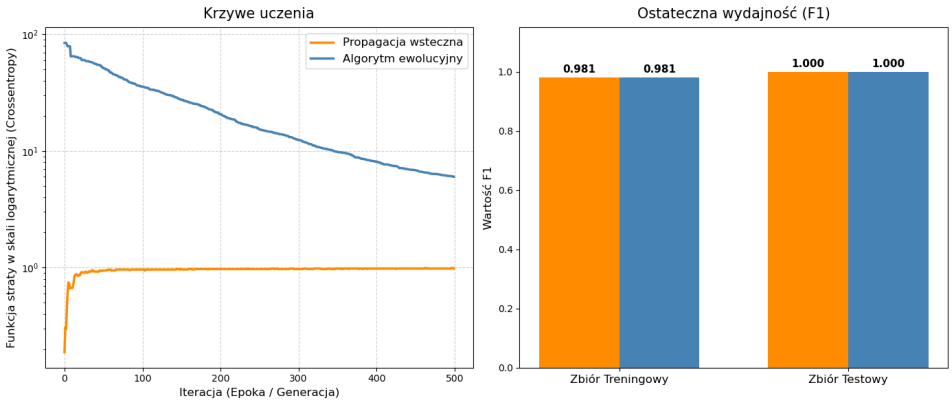


(b) Zbiór auto-mpg

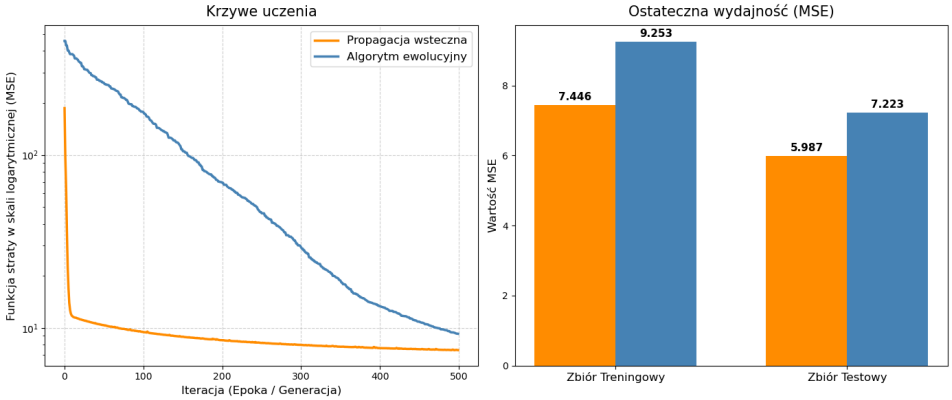


(c) Zbiór multimodal-large

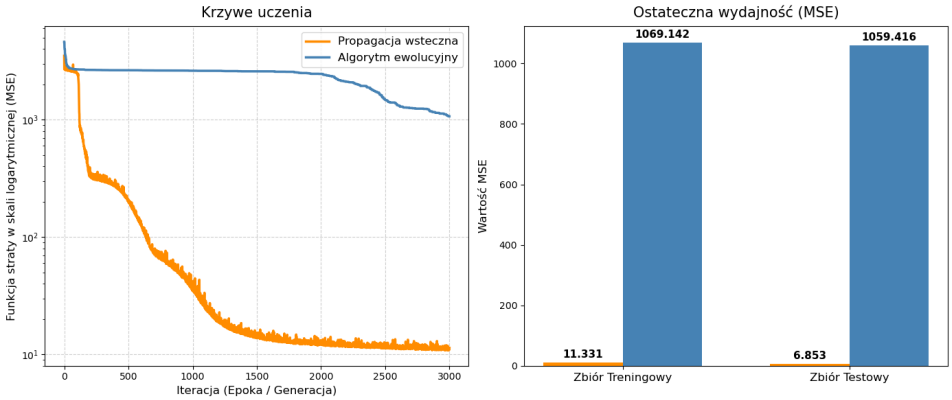
Rysunek 11: Rozkład wyników algorytmu ewolucyjnego i propagacji wstecznej dla danych zbiorów treningowych i testowych.



(a) Zbiór iris



(b) Zbiór auto-mpg



(c) Zbiór multimodal-large

Rysunek 12: Wartość funkcji straty dla kolejnej epoki/generacji oraz ostateczna wydajność dla algorytmu ewolucyjnego i propagacji wstecznej.

Można zauważyć, że w każdym z badanych przypadków algorytm propagacji wstecznej wykazał się szybszą zbieżnością oraz osiągnął lepsze wyniki niż algorytm ewolucyjny. Przewaga ta jest widoczna zwłaszcza dla zbioru danych multimodal-large. Wynika to z faktu, że propagacja wsteczna analitycznie oblicza gradient błędu, precyzyjnie wskazując kierunek poprawy wag, podczas gdy algorytm ewolucyjny przeszukuje wielowymiarową przestrzeń rozwiązań za pomocą losowych mutacji.

Wnioski i podsumowanie

Podsumowując przeprowadzone eksperymenty, zaimplementowany algorytm ewolucyjny udowodnił swoją użyteczność jako wydajne narzędzie do optymalizacji. Przeprowadzona analiza wyników dla różnych architektur i zastosowań pozwala na sformułowanie wniosków.

Algorytm dobrze sprawuje się w znajdowaniu najmniejszej wartości funkcji trójwymiarowej oraz funkcji Rastrigina. Największy wpływ na jakość wyników miał rozmiar populacji - większa wartość ułatwiała szersze przeszukanie prze-

strzeni rozwiązań. Prawdopodobieństwa mutacji oraz krzyżowania w nieco mniejszym stopniu wpływały na ostateczne wyniki, regulując głównie dynamikę zbieżności. Selekcja ruletkowa wprowadzała większą losowość i eksplorację przestrzeni, ale to selekcja rankingowa okazała się najbardziej skuteczna.

Algorytm ewolucyjny pozwolił również na rozwiązanie problemu cutting stock, czyli upakowania okręgu prostokątami. Pozwolił na osiągnięcie optymalnych wyników zarówno pod względem wartościowym, jak i wizualnym. Wprowadzenie inicjalizacji zależnej od stosunku wartości do pola prostokątów dało modelowi ogromną przewagę na starcie. W efekcie już początkowe pokolenia osiągały wysoce satysfakcjonujące rezultaty, przyspieszając zbieżność całego algorytmu.

Ostatnim testowanym zastosowaniem było użycie algorytmu ewolucyjnego do optymalizacji wag w sieci neuronowej MLP. Był on w stanie osiągać zadowalające wyniki, jednak klasyczna nauka analityczna poprzez propagację wsteczną okazała się metodą skuteczniejszą, wykazującą wyższe tempo zbieżności i precyzję.

Eksperymenty te potwierdzają, że algorytmy ewolucyjne to uniwersalne metody, które doskonale radzą sobie w trudnych, nieliniowych i kombinatorycznych przestrzeniach. Jednak w przypadkach, gdzie funkcja celu jest gładka, różniczkowalna i znany jest jej gradient (jak w architekturach MLP), klasyczne metody analityczne pozostają skuteczniejszym podejściem.

Zakres użycia LLM

W trakcie realizacji projektu oraz przygotowywania sprawozdania wykorzystano model językowy Gemini. Służył on jako wsparcie w procesie korekty stylistycznej i gramatycznej tekstu. Dodatkowo model wykorzystano do konsultacji programistycznych przy optymalizacji i debugowaniu tworzonego kodu.